MAD-2 Project Report

Student Details

Name: Nishant Chandra
Roll Number: 23F3000810
Email: 23f3000810@ds.study.iitm.ac.in

About Me: I am a second-year BS student in Data Science and Applications at IIT Madras, with a strong interest in web application development, machine learning, model building, and backend technologies. I enjoy building practical applications that solve real-world problems. Through this project, I aim to strengthen my understanding of full-stack development and develop efficient, scalable solutions that address real-world requirements.

Project Details

Project Title: Placement Portal Application

Problem Statement:
To design and develop a web-based application that streamlines the placement process for students, companies, and administrators.

Approach:
The application was developed using Flask and Vue.js, with SQLite and SQLAlchemy for database management. JWT authentication and role-based access control were implemented for secure user management, while Redis and Celery handle caching and background tasks. Flask-Mail is used for email notifications.

AI/LLM Declaration

I used ChatGPT (GPT-5) as an assistive tool during the development of the project for understanding technical concepts, debugging errors, reviewing code, generating sample data and SQL queries, and improving documentation. It was also used for guidance while working with Flask, Vue.js, SQLAlchemy, Redis, and Celery, as well as understanding implementation approaches for authentication, caching, background jobs, and email notifications.

The extent of AI/LLM usage was approximately 15–20%, primarily limited to technical guidance, code suggestions, debugging support, and documentation assistance. The final implementation, integration, testing, and project decisions were carried out by me.

Technologies and Frameworks Used

Framework/Technology Used	Usage
Flask	Backend web framework for developing REST APIs.
Vue.js	Frontend framework for building the user interface.
SQLite	Database for storing application data.

SQLAlchemy	ORM for database operations and model management.
Flask-JWT-Extended	JWT-based authentication and role-based access control.
Flask-Bcrypt	Secure password hashing.
Flask-CORS	Handling cross-origin requests between frontend and backend.
Redis	Caching and message broker for background tasks.
Celery	Asynchronous and scheduled background job processing.
Flask-Caching	Application-level caching using Redis.
Flask-Mail	Sending emails.
Bootstrap	Frontend styling.
Python-dotenv	Managing environment variables and application configuration.

Database Schema / ER Diagram

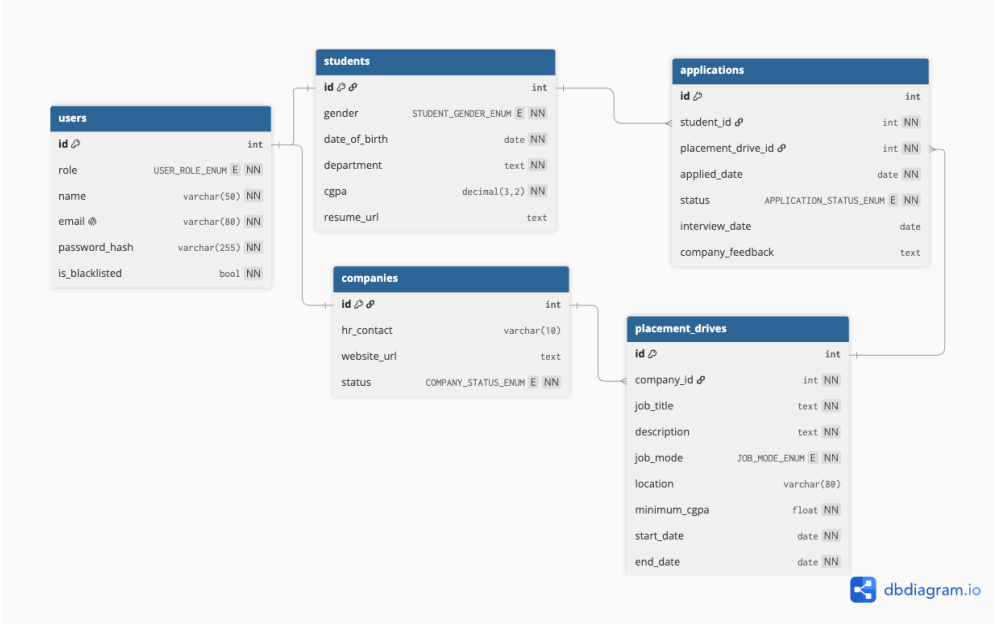
Tables:

1. User: stores common details of all users, including admin, company, and student users.
2. Company: stores company-specific information.
3. Student: stores student-specific information.
4. PlacementDrive: stores placement drive details.
5. Application: stores application details.

Relationships:

- One-to-One → User → Company
- One-to-One → User → Student
- One-to-Many → Company → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application

ER Diagram: Click Me to view in more detail.



API Resource Endpoints

Endpoint	Method	Description
`/api/v2/auth/tokens`	POST	Authenticate user and generate a JWT access token
`/api/v2/auth/companies`	POST	Register a new company
`/api/v2/auth/students`	POST	Register a new student
`/api/v2/auth/me`	GET	Fetch details of the currently authenticated user
`/api/v2/admin/stats`	GET	Fetch admin dashboard statistics
`/api/v2/admin/companies`	GET	Fetch companies for admin management
`/api/v2/admin/companies/{company_id}`	PATCH	Approve or reject a company
`/api/v2/admin/students`	GET	Fetch students for admin management
`/api/v2/admin/users/{user_id}`	PATCH	Update a user's blacklist status
`/api/v2/admin/placement-drives`	GET	Fetch placement drives for admin management
`/api/v2/admin/placement-drives/{placement_drive_id}`	PATCH	Update or complete a placement drive
`/api/v2/admin/applications`	GET	Fetch placement applications
`/api/v2/admin/students/{student_id}`	GET	Fetch details of a specific student
`/api/v2/company/placement-drives`	POST	Create a new placement drive
`/api/v2/company/placement-drives`	GET	Fetch the company's placement drives
`/api/v2/company/placement-drives/{placement_drive_id}`	PATCH	Update a placement drive

`/api/v2/company/applications`	GET	Fetch applications for the company's placement drives
`/api/v2/company/students/{student_id}`	GET	Fetch details of a student
`/api/v2/company/applications/{application_id}`	PATCH	Update application status, interview date, or feedback
`/api/v2/company/export-applications`	POST	Start background export of company applications
`/api/v2/student/`	GET	Fetch the authenticated student's profile
`/api/v2/student/`	POST	Update the authenticated student's profile
`/api/v2/student/companies`	GET	Fetch companies available to the student
`/api/v2/student/placement-drives`	GET	Fetch placement drives available to the student
`/api/v2/student/applications/`	POST	Submit an application for a placement drive
`/api/v2/student/applications`	GET	Fetch the student's applications
`/api/v2/student/export-applications`	POST	Start background export of student applications
`/api/v2/common/celery-task-status/{task_id}`	GET	Check the status of a background Celery task
`/<filename>`	GET	Download a generated file

Video Presentation

Link: project_demo.mp4